

Thorpe Desktop — The Solution Offered

Prepared for: Patent counsel

Product: Thorpe Desktop

Core invention: Jonathan — autonomous AI technician with local repair orchestration

Date: June 27, 2026

Software version: 1.1.0

Companion document: 10-PROBLEMS-SOLVED-FOR-COUNSEL.md

Executive summary

Thorpe Desktop's solution is **Jonathan**, an embedded AI technician that operates as a **local-first incident orchestration system** on the end-user device. Jonathan does not merely generate advice—it **plans, executes, verifies, and then narrates** IT repairs through a governed pipeline that runs entirely on Windows, macOS, or Linux.

The architectural principle that defines the invention: **execute-then-narrate**. All diagnostics and approved repairs complete **before** any AI-generated explanation reaches the user. The narrative layer—whether local template or cloud LLM—is constrained to describe only repairs that were actually performed and recorded.

What Thorpe Desktop is

Thorpe Desktop is a cross-platform desktop application built with **Tauri** (Rust backend + React frontend). It provides:

- **Jonathan Assistant** — conversational IT support with autonomous local repair
- **System Scanner** — health scoring and issue detection
- **Repair Center** — explicit user-initiated maintenance with consent
- **Proactive Watchdog** — background CPU, memory, and disk monitoring with handoff to Jonathan
- **Offline connectivity diagnostics** — layered network troubleshooting without cloud
- **Enterprise controls** — signed repair packs, licensing, AI policy, audit trails

Jonathan is the unifying agent that ties these subsystems into a single incident pipeline.

The solution architecture

High-level pipeline

- User message (or Watchdog alert)
- Evidence collection
 - Security escalation gate
 - Optional offline connectivity suite
 - Repair planning (LLM JSON or rules)
 - Signed repair-pack allowlist filter
 - Confirmation gate (mutating vs. diagnostic)
 - Local OS repair execution
 - Post-mutating verification scan
 - Session persistence + optional PSA webhook
 - Narrative generation (local or cloud)
 - Structured chat response

Layered system design

Layer	Role	Key modules
Presentation	Chat UI, approvals, notifications	JonathanAssistant.tsx, AppLayout.tsx
API	Tauri commands bridge UI to Rust	chat_with_jonathan, execute_repair
Orchestration	Incident pipeline owns execution order	orchestrate_incident (agent/mod.rs)
Planning	LLM JSON plan with rule fallback	build_plan, plan_with_llm, plan_with_rules
Repair engine	OS command execution + history	repairs/mod.rs, perform_repair
Narrative	Post-execution explanation only	format_technician_response, cloud LLM
Persistence	SQLite audit trail	chat history, agent sessions, evidence artifacts
OS	Platform commands	PowerShell, shell, ping, DNS

Critical invariant: Cloud LLM failure does not prevent repairs from having already executed locally.

How the solution works — step by step

Step 1 — User initiates an incident

The user describes a problem in natural language (e.g., "Wi-Fi isn't working", "my computer is slow") or clicks **Send to Jonathan** from a Watchdog alert. The frontend calls chat_with_jonathan with the message, optional scan context, chat history, and any user-approved repair IDs.

Step 2 — Evidence collection

orchestrate_incident collects local system evidence: event log excerpts, service summary, network summary, and scan snapshot. Evidence is persisted as artifacts linked to the session for audit and planner context.

Step 3 — Security gate

If the message contains malware, credential, or other high-risk keywords, the system creates a PSA escalation case and **does not execute repairs**. This prevents unsafe autonomous action on security-sensitive incidents.

Step 4 — Offline connectivity suite (when applicable)

For network-related issues, Jonathan runs a layered connectivity diagnostic **before planning**:

- 1. Network adapter status
- 2. Default gateway reachability
- 3. DNS resolution
- 4. Internet reachability

A local playbook evaluates results and produces `overall_status`, recommended repair actions, and a human-readable summary—all **without cloud connectivity**. Results are saved and embedded in the chat response. Duplicate connectivity diagnostics are suppressed in the subsequent repair plan.

Step 5 — Repair planning

Jonathan builds an AgentPlan using a **dual planner**:

- **LLM planner** (when enterprise or user cloud runtime available): JSON-structured plan with hypotheses, confidence, tool steps, risk levels, and approval requirements
- **Rule planner** (fallback): keyword and scan-signal mapping to repair tool IDs

If the LLM is unavailable or returns empty steps, rules take over. Planning never blocks execution.

Step 6 — Signed repair-pack allowlist

Planned tool IDs are intersected with `allowed_tool_ids` from enabled **cryptographically signed repair packs** (Ed25519). The planner cannot invoke OS tools outside the allowlist. Built-in packs ship with the app; custom packs are installable via signed manifests.

Step 7 — Confirmation gate

Repair actions are classified:

Kind	Behavior	Examples
Diagnostic	Auto-execute	connectivity suite, disk analysis, resource ID
Mutating	Require user approval	DNS flush, temp cleanup, spooler restart
Advisory	Informational only	startup review, update check, restart recommend

Safe diagnostics run immediately. Mutating repairs appear as **pending approval** until the user clicks Approve.

Step 8 — Local repair execution

`perform_repairs_with_failures` executes each approved action via platform-specific commands. Every result—success or failure—is recorded in `repair_history` with action kind, message, and details.

Step 9 — Post-mutating verification

When any mutating repair succeeds, Jonathan runs `quick_system_scan()` and produces structured `RepairVerification`:

- Health score before and after
- Issue count before and after
- `improved` flag when health increased or issues decreased

Verification is surfaced in the UI and appended to both local and cloud narratives.

Step 10 — Narrative generation (execute-then-narrate)

Only after steps 1–9 complete, Jonathan generates the user-facing reply:

- **Local mode** (`format_technician_response`): deterministic markdown—no network required
- **Cloud mode** (`call_openai_compatible`): conversational LLM with system prompt stating repairs **already ran**; LLM must use past tense and must not invent actions
- **Enterprise mode**: governed by org policy, budget, role, and provider configuration

Step 11 — Structured response

The user receives a single `ChatResponse` containing:

- Narrative text
- Repairs executed (grouped by kind)
- Pending repairs awaiting approval
- Verification metrics
- Connectivity report (if applicable)
- Agent plan and knowledge suggestions
- Optional escalation case ID

All metadata is persisted in `metadata.json` for session restore.

Solution components in detail

A. Execute-then-narrate autonomous agent (Patent A)

What it solves: Chatbots that describe fixes they never performed.

How Thorpe solves it: `chat_with_jonathan` calls `orchestrate_incident()` **before** any narrative generation. The LLM is a narrator constrained by executed repair records, not an unconstrained actor.

Code anchor: `src-tauri/src/agent/mod.rs` — `orchestrate_incident`

B. Signed repair-pack gating (Patent B)

What it solves: Uncontrolled AI invocation of arbitrary OS commands.

How Thorpe solves it: Repair packs contain signed manifests listing permitted tools.

`filter_allowed_tools` intersects planner output with the allowlist before any command runs. Built-in packs (`thorpe-core`, `thorpe-network`, `thorpe-performance`) ship with the application; operators can add custom signed packs.

Code anchor: `src-tauri/src/repairs/pack_signing.rs`, `filter_allowed_tools`

C. Offline connectivity fusion (Patent C)

What it solves: Cloud-dependent network troubleshooting when the network itself is the problem.

How Thorpe solves it: Keyword-triggered pre-run of layered local checks, local playbook decision tree, persistence as session evidence, deduplication in repair plan, and embedding in chat response—all offline-capable.

Code anchor: `src-tauri/src/connectivity/mod.rs`

D. Proactive Watchdog handoff (Patent D)

What it solves: Reactive-only support; monitoring siloed from repair.

How Thorpe solves it: Background loop (default 15-minute interval) monitors CPU, memory, disk, and health score. Per-metric alerts dedupe within the interval. User clicks **Send to Jonathan**; a synthetic prompt feeds the **same** `orchestrate_incident` pipeline as user chat, with optional pre-loaded rule plan.

Code anchor: `src-tauri/src/watchdog/mod.rs`, `src/lib/watchdog.ts`

E. Dual LLM with independent fallbacks (Patent E)

What it solves: Single-point-of-failure when one LLM handles both planning and narration.

How Thorpe solves it: Separate planner LLM (JSON, temperature 0.2) and narrator LLM (conversational, temperature 0.7), each with rule-based or local-template fallbacks. Planner failure does not block repairs; narrator failure does not undo executed repairs.

Code anchor: `src-tauri/src/agent/planner.rs`, `src-tauri/src/ai/mod.rs`

User experience of the solution

For the end user

1. Open Jonathan and describe the problem in plain language.
2. Jonathan runs diagnostics automatically and shows what it found.
3. For system-changing fixes, Jonathan asks **Approve** before acting.
4. After repairs, Jonathan shows whether health improved.
5. Everything works offline for core diagnostics and local explanations.

For IT / MSP (optional)

- PSA webhook escalation for security or low-confidence cases
- Enterprise AI console with role, budget, and provider policy
- Intelligence console with org playbooks, repair pack management, session audit
- PDF export of incident sessions

For counsel — reduction to practice

Claim	Shipped evidence
Local execution	Rust repair engine runs OS commands on device
Execute-then-narrate	orchestrate_incident precedes narrative in chat_with_jonathan
Verification	RepairVerification struct with before/after health
Signed packs	Ed25519 verification on install_repair_pack
Offline connectivity	run_connectivity_suite + playbook without network
Watchdog handoff	WatchdogHandoff → synthetic prompt → same pipeline
Platforms	Windows, macOS, Linux (Tauri)
Tests	70+ frontend, 40+ Rust unit tests

Solution vs. prior art (summary)

Prior art category	Their approach	Thorpe solution
IT chatbots	Manual step recommendations	Local OS repair execution
RMM/MSP tools	Central-managed scripts	User-initiated NL on device
OS troubleshooters	Fixed decision trees	LLM + rules hybrid planner
LLM tool agents	Cloud interleaved plan/act	Deterministic Rust engine; execute-then-narrate

Utility apps	Single-purpose (DNS flush, etc.)	Unified incident session with evidence + verification
--------------	----------------------------------	---

Recommended patent claims from this solution

1. **Method** for receiving NL support request, planning repairs locally, filtering against signed allowlist, executing diagnostics automatically, deferring mutating repairs, verifying with follow-up scan, and generating narrative only after execution.
2. **System** comprising planner, repair engine, allowlist module, verification module, and narrator constrained to executed repairs.
3. **Method** for offline connectivity pre-run fused into agent incident with deduplication.
4. **Method** for proactive monitor alert handoff into same orchestration pipeline as user chat.

Full claim seeds: 04 - CLAIM - SEEDS - FOR - ATTORNEY .md

One-sentence solution statement

Thorpe Desktop offers a governed, local-first AI technician that plans and executes IT repairs on the client device, verifies outcomes with a follow-up health scan, and only then explains what it did—ensuring the AI cannot claim fixes it did not perform.

Related documents

Document	Content
01 - INVENTION - DISCLOSURE - JONATHAN .md	Full technical disclosure
03 - SYSTEM - ARCHITECTURE - AND - FLOWS .md	Architecture diagrams
04 - CLAIM - SEEDS - FOR - ATTORNEY .md	Seed claims
10 - PROBLEMS - SOLVED - FOR - COUNSEL .md	Problems brief (companion)

Applicant / assignee: _[Complete Document 06 before filing]_